03

GitOps 구축

서버에 SSH로 산출물을 보내던 배포를 EKS 위 ArgoCD 기반 GitOps 배포로 전환하고, 서비스마다 달랐던 Jenkins 파이프라인을 공용 라이브러리로 통일

역할	플랫폼 설계와 구축, 파이프라인 표준화
환경	EKS, Terraform, Jenkins, Nexus, ArgoCD, Helm, External Secrets
결과	신규 서비스 배포에 추가하는 설정 파일 2개 Shared Library 원자 함수 7개, 조합 템플릿 4개 Terraform 커스텀 모듈 15종 프론트엔드 이미지 1.09GB에서 292MB로 축소 (Multi-stage)

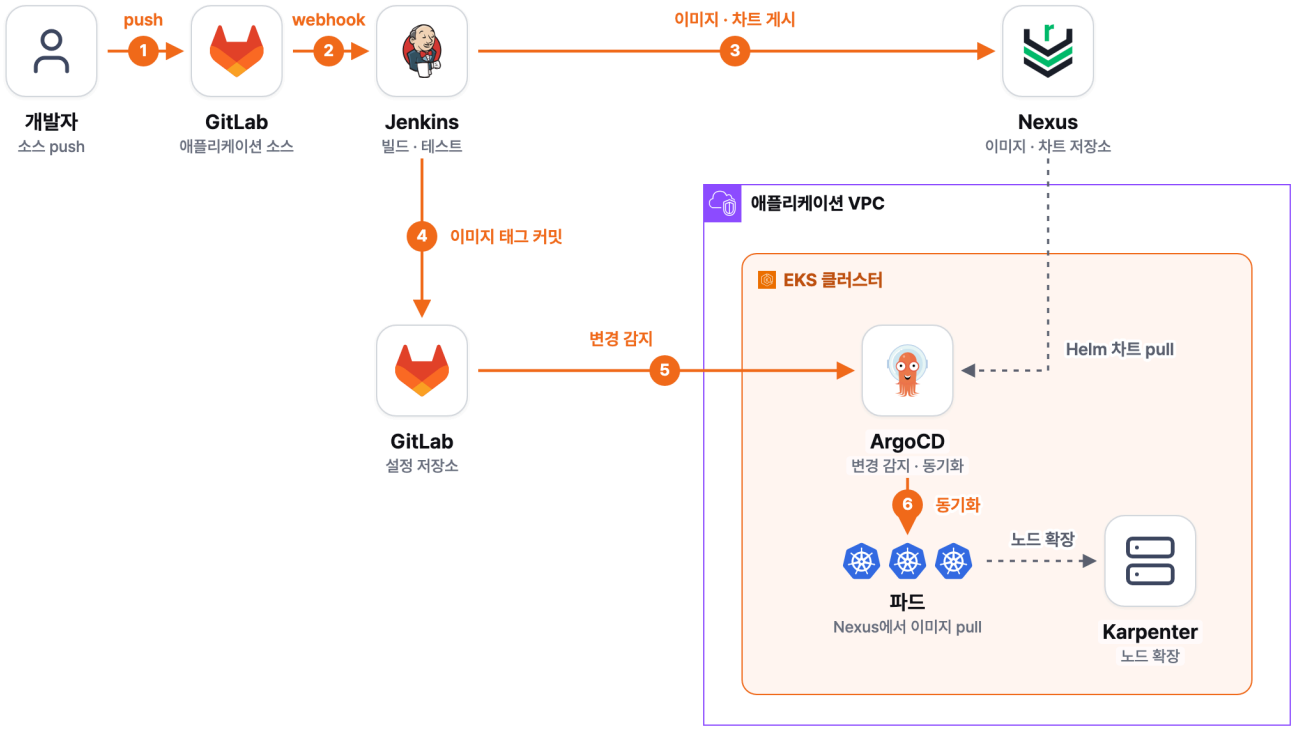
배경과 배포 흐름

기존 방식

- Jenkins가 SSH로 서버에 산출물을 보내는 Push 방식
- 서비스마다 파이프라인이 달라 어떤 버전이 어디에 있는지 Git으로 추적 불가
- 설정 파일에 DB 비밀번호와 AWS 키가 커밋된 서비스 존재

전환 기준

- Jenkins는 빌드와 Git 커밋까지만, 클러스터 반영은 ArgoCD 전담
- Git이 배포 기록이므로 롤백은 커밋 revert
- AWS는 Terraform, 쿠버네티스 안은 ArgoCD로 경계 분리



1 소스 push
개발자가 애플리케이션 소스를 GitLab에 push

2 CI 트리거
GitLab webhook이 Jenkins를 실행하고, 공용 라이브러리 파이프라인으로 빌드·테스트

3 산출물 게시
Docker 이미지와 공통 라이브러리, Helm 차트를 Nexus에 게시

4 태그 커밋
Jenkins가 설정 저장소의 이미지 태그를 갱신하고 커밋

5 변경 감지
EKS 안의 ArgoCD가 설정 저장소의 변경을 감지하고, Nexus에서 차트를 pull

6 클러스터 동기화
매니페스트를 적용하면 Karpenter가 필요한 노드를 늘리고, 파드가 이미지를 pull

설계 선택

App-of-Apps

- root Application이 apps 디렉터리를 재귀 등록. 신규 서비스는 `Application.yaml`과 `values.yaml` 추가로 끝
- ApplicationSet 대신 개별 Application 선택. 서비스마다 네임스페이스, 차트 버전, values 구조가 다름
- root를 앱용(prune 사용)과 플랫폼용(prune 미사용)으로 분리. AppProject로 개발자와 CI 계정은 앱 범위만 접근

Jenkins Shared Library

- 원자 함수 7개: checkout, 이미지 빌드, push, 이미지 정리, GitOps values 갱신, ArgoCD sync, 알림. 함수끼리 호출하지 않음
- 조합 템플릿 4개: QA 자동 배포, 운영 빌드, 운영 배포(수동 승인), 수동 검증
- 운영은 Build와 Deploy Job 분리. Deploy가 실행 시점의 Nexus 최신 태그를 조회하고 승인 게이트 통과 후 진행

시크릿과 설정값 분리

- 시크릿은 Secrets Manager와 External Secrets, 설정값은 GitOps values의 ConfigMap
- AWS 자격 증명은 키 없이 EKS Pod Identity. IRSA와 달리 공용 라이브러리 수정 없이 적용
- 설정 파일의 평문 기본값을 6개 서비스에서 제거, 정적 액세스 키 문자열 0건 확인

Terraform과 ArgoCD 관리 경계

쿠버네티스 API 밖이면 Terraform, 안이면 ArgoCD. 예외는 부트스트랩(ArgoCD 자신, Karpenter). Traefik, External Secrets, ALB Controller, Gateway API CRD를 Terraform에서 ArgoCD로 이관.